



C++ Programming Language

Presented By

Usha Rani Sargurunathan

STG Group

Type Inference

auto Keyword

Definition: The auto keyword allows the compiler to deduce the type of a variable from its initializer at compile time.

Purpose:

Reduces verbosity, especially for complex or long type names (e.g., iterator types, std::map key-value pairs).

Improves code readability by focusing on the variable's purpose rather than its exact type.

Simplifies code refactoring, as type changes in the initializer don't require manual type updates.

Syntax: `auto variable_name = initializer;`

Example - auto

```
// int main() {  
//     auto i = 10;           // i is int  
//     auto d = 3.14;         // d is double  
//     auto s = "hello";      // s is const char*  
//     return 0;  
// }
```

Lambda Expressions

Lambda Expressions

A **lambda expression** is a way to define an **anonymous** (nameless) function object right where you need it. Lambdas are particularly useful for passing a function as an argument to another function, such as with standard library algorithms.

The basic syntax includes:

[capture_clause] (parameters) -> return_type { body }

- **[] (Capture Clause)**: Specifies variables from the surrounding scope that the lambda can access.
- **() (Parameter List)**: The arguments the lambda takes.
- **{ } (Function Body)**: The code to be executed.

Lambda Syntax

Lambda Expression in C++

start of the
lambda

Return Type

```
[](int a) -> int { return a - 1; };
```

Parameter
List

Body

The diagram illustrates the syntax of a C++ lambda expression. A green rounded rectangle contains the code `[](int a) -> int { return a - 1; };`. Four arrows point from labels to parts of the code: 'start of the lambda' points to the opening square bracket, 'Return Type' points to the `-> int` part, 'Parameter List' points to the `(int a)` part, and 'Body' points to the `{ return a - 1; }` part.

Key Concept: The Capture Clause

- The biggest differentiator between a regular function and a lambda is the **capture clause**, which enables a lambda to "remember" variables from its surrounding code context:
- **[]** – Captures nothing. It can only use its parameters.
- **[=]** – Captures all external variables used inside the body **by value** (makes a copy).
- **[&]** – Captures all external variables used inside the body **by reference** (can modify the originals).
- **[x, &y]** – Mixed capture. Captures x by value and y by reference.

Lambda Parameters

Definition: The input arguments that a lambda expression accepts. They are specified in parentheses, similar to regular function parameters.

Characteristics:

- Can be explicitly typed (e.g., (int x, double y)).
- Can use auto for generic parameters (C++14 onwards: (auto x, auto y)).
- Can have default arguments (C++20 onwards).
- Can be variadic (C++11 onwards with variadic templates).

Purpose: To define the signature of the anonymous function.

Example:

```
// auto sum = [](int a, int b) { return a + b; };  
// auto generic_sum = [](auto a, auto b) { return a + b; }; // C++14 generic lambda  
// int result = sum(5, 3);  
/// double result_gen = generic_sum(5.5, 3.2);
```

Lambda Body

Definition: The block of code enclosed in curly braces ({}) that defines the operations performed by the lambda expression.

Characteristics:

- Can contain any valid C++ statements.
- Can access variables from its surrounding scope through the capture list.
- Can include local variable declarations.
- Can have return statements.

Purpose: To implement the logic of the anonymous function.

Example:

```
// int x = 10;  
  
// auto printer = [x]() {  
//     // std::cout << "Value of x: " << x << std::endl;  
// };  
  
// printer();
```

Storing Lambdas

Concept: Lambdas are objects (specifically, unique, unnamed closure types) and can be stored in variables, passed as arguments, or returned from functions.

Mechanisms for Storage:

auto: The simplest way to store a lambda, as auto will deduce its unique closure type.

std::function: A type-safe wrapper (from the `<functional>` header) that can store, copy any callable object (including lambdas, function pointers, and function objects) that matches a specific signature.

Function Pointers: Only possible for lambdas that do not capture any variables and have a compatible signature.

Example - Storing Lambdas

```
#include <functional> // For std::function

// int main() {
//     auto lambda_auto = [](int a, int b) { return a + b; }; // Stored with auto
//     std::function<int(int, int)> lambda_func = [](int a, int b) { return a * b; };
//     Stored with std::function
//
//     // int result_auto = lambda_auto(2, 3);
//     // int result_func = lambda_func(2, 3);
//     return 0;
// }
```

References to Outside Context (Closures)

- **Concept:** A lambda expression, when it captures variables from its surrounding scope, forms a "closure." The closure is an object that contains the lambda's code and a copy or reference to the captured variables.
- **Purpose:** To allow anonymous functions to "close over" their environment, making them more powerful and context-aware.
- **Capture List ([]):** The mechanism to specify which variables from the enclosing scope are captured and how.
 - []: No capture.
 - [var]: Capture **var** by value.
 - [&var]: Capture **var** by reference.
 - [=]: Default capture all used variables by value.
 - [&]: Default capture all used variables by reference.
 - [this]: Capture this pointer by value (for class members).

Example – 1 (Function call)

```
#include <iostream>

using namespace std;

int main()
{
    [] () { cout << "Hello, World!" << endl; }();
}
```

Example – 2 (Taking parameters)

```
#include <iostream>
using namespace std;
int main() {
    [] (int x, int y) {
        cout << "Sum is: " << x + y << endl;
    } (5, 10);
}
```

Example – 3 (Print sum)

```
#include <iostream>

using namespace std;

int main() {
    cout<<([] (int x, int y) { return x + y; }(10, 30))<<endl;
}
```

Example – 4 (Print variable)

```
#include <iostream>

using namespace std;

int main() {
    int a= [](int x, int y)-> int{ return x + y; }(10, 30);
    cout << "Sum is: " << a << endl;
}
```

Example – 5 (Access Local Variable)

```
#include <iostream>
using namespace std;
int main() {
    int a= 10;
    [a]() {cout<<a;} ();
}
```

Example – 5 (Assign Value)

```
#include <iostream>
using namespace std;
int main() {
    int a= 10;
    auto f=[a]() {cout<<a<<endl;};
    f();
    f();
}
```

Example – 6 (Pass by Value and Reference)

```
#include <iostream>
using namespace std;
int main() {
    int a= 10;
    auto f=[a]() {cout<<a<<endl;};
    f();
    a++;
    f();
}
```

```
#include <iostream>
using namespace std;
int main() {
    int a= 10;
    auto f=[&a]() {cout<<a<<endl;};
    f();
    a++;
    f();
}
```

Example – 7 (Pass Lambda as Parameter)

```
#include <iostream>
using namespace std;
template <typename T>
void fun(T p) {
    p();
}
int main() {
    int a=10;
    auto f=[&a]() {cout<<a++<<endl;};
    fun(f);
    fun(f);
}
```

Tasks

- Write a lambda expression that prints "Welcome to C++ Programming".
- Write a lambda expression that returns the larger of two numbers.
- Write a lambda expression that checks whether a number is even or odd.

Dangling Pointers

Dangling Pointers

Definition: A pointer that points to a **memory location that has been deallocated** (or to an object that no longer exists).

Problem: Dereferencing a dangling pointer leads to undefined behavior, which can manifest as:

- Program crashes (segmentation fault).
- Corrupted data.
- Security vulnerabilities.
- Subtle, hard-to-debug errors.

Causes:

- Deleting memory pointed to by a raw pointer, but the pointer itself is not set to **nullptr**.
- Returning a pointer to a local variable that goes out of scope.
- Multiple raw pointers pointing to the same memory, and one of them deletes it.

Prevention:

- Always set raw pointers to **nullptr** immediately after **deleting** the memory they point to.
- Prefer smart pointers (**std::unique_ptr**, **std::shared_ptr**) as they automate memory management and reduce dangling pointer issues.
- Avoid returning pointers or references to local stack variables.

Returning Reference

```
#include <iostream>
using namespace std;
int somefn();
int main() {
    int x;
    x= somefn();
    cout<<x; }
```

```
int somefn() {
    int a=10;
    return a; }
```

```
#include <iostream>
using namespace std;
int& somefn();
int main() {
    int x;
    x= somefn();
    cout<<x; }
```

```
int& somefn() {
    static int a=10;
    return a; }
```

Returning Reference

```
#include <iostream>
using namespace std;
int& somefn();
int main() {
    int x;
    somefn()=15;
    cout << somefn() << endl;
}
int& somefn() {
    static int a =10;
    return a;
}
```

Smart Pointers

Smart Pointers

Smart pointers are objects that act like pointers but automate memory management.

They are C++'s modern solution to preventing memory leaks and dangling pointers by wrapping raw pointers with ownership semantics.

The memory they manage is automatically deallocated when the smart pointer object is destroyed.

unique_ptr

A **unique_ptr** is a smart pointer that owns the object it points to.

Its ownership is **exclusive**, meaning no two **unique_ptr**s can point to the same object at the same time.

This pointer cannot be copied; it can only be moved.

A **unique_ptr** is the best choice for non-shared, dynamically allocated objects.

It's lightweight and has virtually no overhead.

shared_ptr

A **shared_ptr** is a smart pointer that allows **shared ownership** of an object. Multiple **shared_ptr**s can point to the same object. The **shared_ptr** internally maintains a **reference count**.

The object is automatically deallocated only when the last **shared_ptr** pointing to it is destroyed or reset. The reference count increases when a new **shared_ptr** is created from an existing one, and it decreases when a **shared_ptr** is destroyed.

weak_ptr

A **weak_ptr** is a non-owning smart pointer. It points to an object managed by a **shared_ptr** but does **not** increase the object's reference count. Its main purpose is to break **circular references** between **shared_ptr**s, which would otherwise prevent the memory from ever being deallocated.

You cannot directly access the object through a **weak_ptr**. You must first convert it to a **shared_ptr** by calling its **lock()** method. If the object has already been destroyed, **lock()** returns an empty **shared_ptr**.

Example – Unique Pointer

```
#include<iostream>
#include<memory>
using namespace std;
class Rectangle
{
    int length;
    int breadth;
public:
    Rectangle(int l, int b) {
        length=l;
        breadth=b; }
    int area() {
        return length*breadth; }
};

int main()
{
    unique_ptr <Rectangle> ptr (new Rectangle (10,5));
    cout<<"ptr: "<<ptr->area()<<endl;
    unique_ptr<Rectangle> ptr2;
    ptr2=move(ptr);
    cout<<"ptr2: "<<ptr2->area()<<endl;
    cout<<"ptr: "<<ptr->area()<<endl; // This will
    cause a runtime error since ptr is now null after move
}
```

Example – Shared Pointer

```
# include<iostream>
# include<memory>
using namespace std;
class Rectangle
{
    int length;
    int breadth;
public:
    Rectangle(int l, int b) {
        length=l;
        breadth=b; }
    int area() {
        return length*breadth; }
};
```

```
int main()
{
    shared_ptr<Rectangle> ptr(new
    Rectangle(10,5));
    cout<<ptr->area()<<endl;
    shared_ptr<Rectangle> ptr2;
    ptr2=ptr;
    cout<<"ptr2 "<<ptr2->area()<<endl;
    cout<<"ptr "<<ptr->area()<<endl;
    cout<<ptr.use_count()<<endl;
}
```

Example – Weak Pointer

```
#include <iostream>
#include <memory>

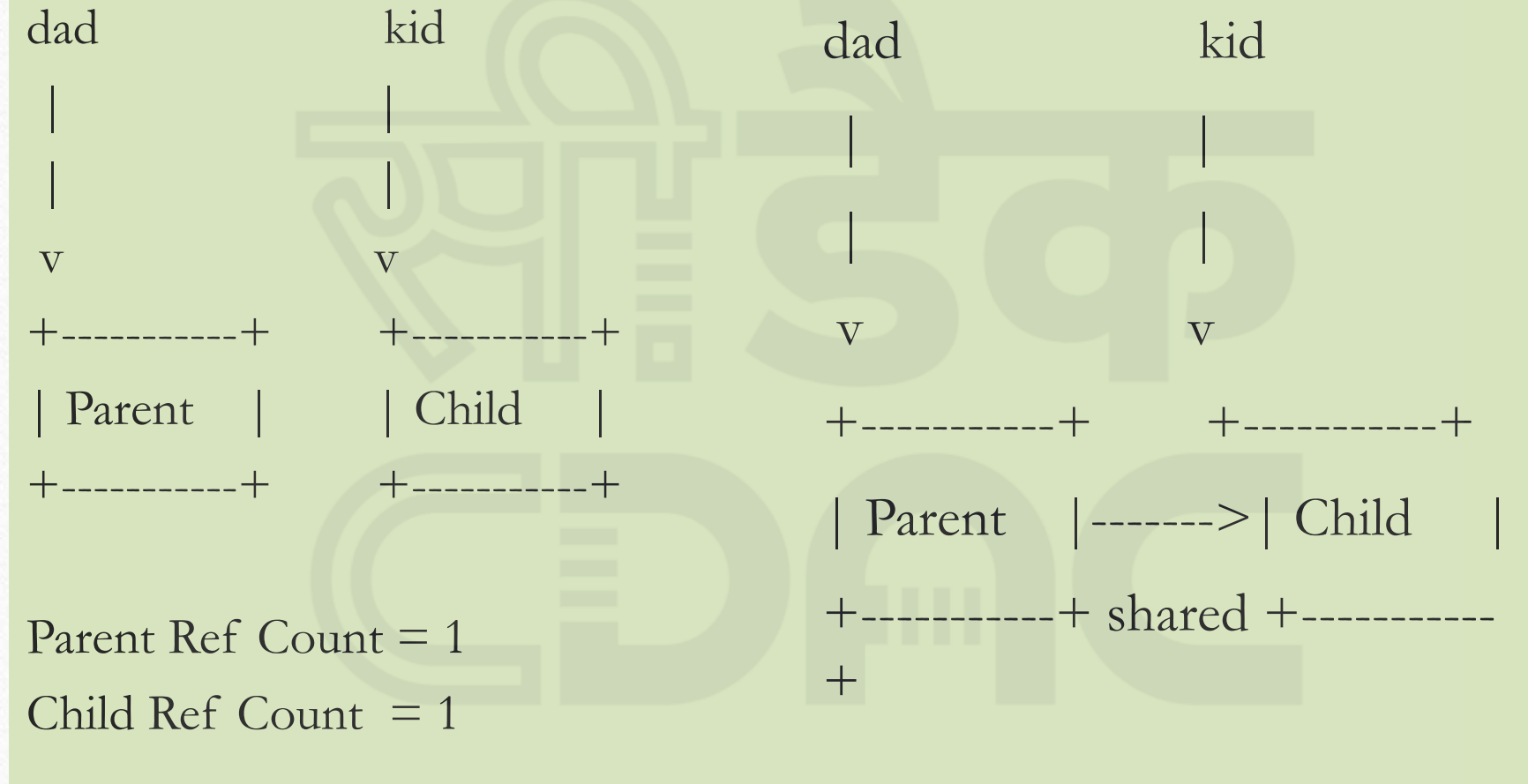
class Child; // Forward declaration

class Parent {
public:
    std::shared_ptr<Child> my_child;
    ~Parent() { std::cout << "Parent is destroyed." <<
std::endl; }
};

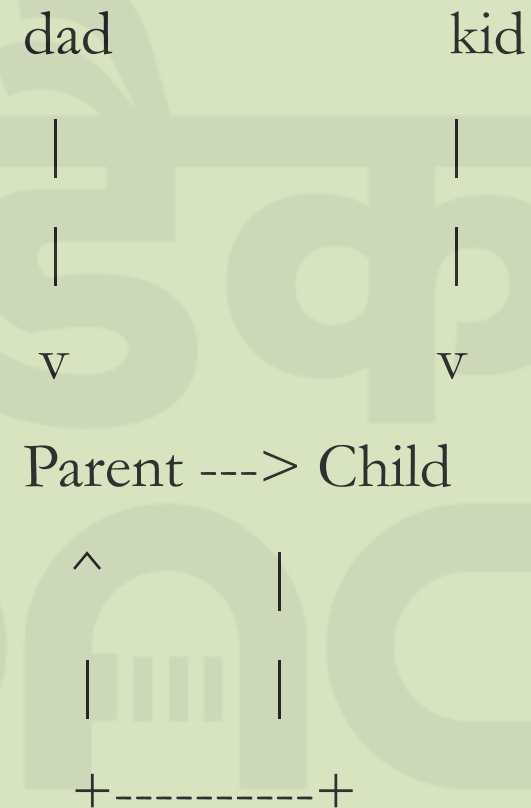
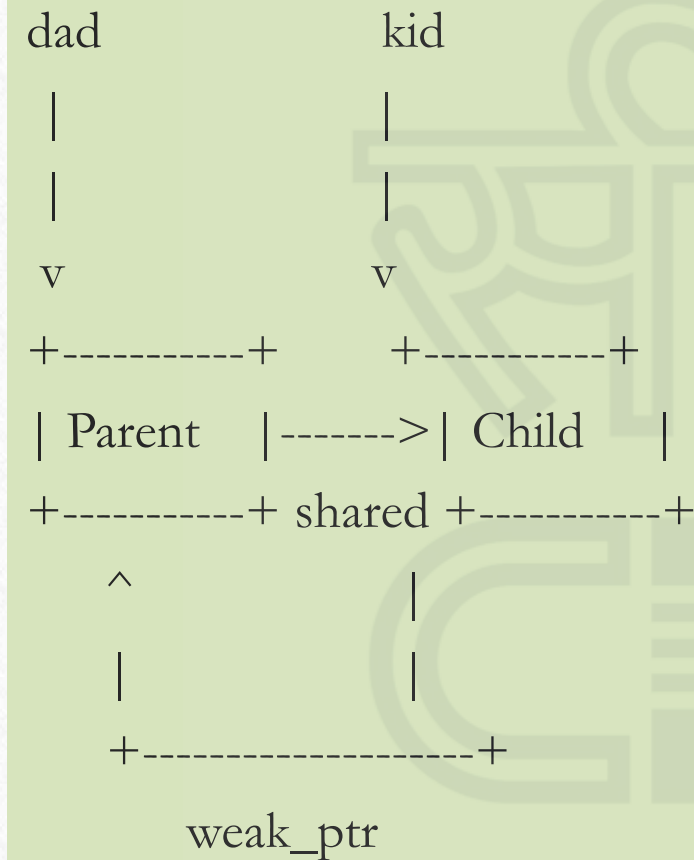
class Child {
public:
    std::weak_ptr<Parent> my_parent; // weak_ptr for
non-owning back reference
    ~Child() { std::cout << "Child is destroyed." <<
std::endl; }
};

int main() {
    std::shared_ptr<Parent> dad = std::make_shared<Parent>(); // ref count
of dad = 1
    std::shared_ptr<Child> kid = std::make_shared<Child>(); // ref count of
kid = 1
    dad->my_child = kid; // ref count of kid = 2
    kid->my_parent = dad; // Weak ptr, no change to dad's ref count
(still 1)
    // At the end of main():
    // dad shared_ptr goes out of scope, ref count of dad drops from 1 to 0.
    Parent is destroyed.
    // When Parent is destroyed, its my_child shared_ptr is also destroyed,
    ref count of kid drops from 2 to 1.
    // The kid shared_ptr goes out of scope, ref count of kid drops from 1 to
    0. Child is destroyed.
    // Using a weak_ptr:
    if (auto parent_ptr = kid->my_parent.lock()) {
        std::cout << "The parent still exists!" << std::endl;
    } else {
        std::cout << "The parent no longer exists." << std::endl; }
    return 0; }
```

Explain



Circular reference (reference cycle)



Accessing the Parent

- A `weak_ptr` cannot be used directly.
- Instead, call `lock()` function.
- If Parent still exists:
 - returns a `shared_ptr<Parent>` `// "Is the Parent still alive? If yes, give me temporary access."`
- If Parent has already been destroyed:
 - returns an empty `shared_ptr`
- because `parent_ptr` is a temporary `shared_ptr`.
- When the if block ends, `parent_ptr` is destroyed, so Parent's reference count goes back to 1.

Key takeaway

- **shared_ptr = ownership.** It increases the reference count and keeps the object alive.
- **weak_ptr = observation.** It does **not** increase the reference count.
- Use **weak_ptr** when two objects need to refer to each other, but only one should own the other.
- Always use **lock()** to safely access the object managed by a **weak_ptr**.
- **weak_ptr** is the standard way to prevent **circular references** and memory leaks when objects have bidirectional relationships.

Const Members

Const Members

The `const` keyword provides a compile-time guarantee that a value will not be modified. Within a class, it applies to both data and functions.

- **Const data members:** A data member declared `const` must be initialized in the constructor's initialization list and cannot be modified afterward.
- **Const member functions:** A function declared `const` promises not to change the object's state. You can only call `const` member functions on a `const` object.

Const Data Members

Definition: A data member declared with the `const` keyword.

Characteristics:

- Must be initialized at the time of object creation (either in the constructor's initializer list or at declaration for static const members).
- Their value cannot be changed after initialization.

Purpose: To represent immutable properties of an object.

Example:

```
class Circle {  
private:  
    const double PI; // Const data member  
    double radius;  
public:  
    Circle(double r) : PI(3.14159), radius(r) {} // Initialize in initializer list  
};
```

Const Member Functions

Definition: A member function declared with the `const` keyword after the parameter list.

Characteristics:

- Guaranteed not to modify any non-`mutable` data members of the object.
- Can be called on both `const` and non-`const` objects.
- Cannot call non-`const` member functions of the same object.

Purpose: To provide read-only access to an object's state and to enable calling functions on `const` objects.

Example:

```
class Point {  
    int x, y;  
public:  
    Point(int px, int py) : x(px), y(py) { }  
    int getX() const // Const member function  
        { return x; }  
    // void setX(int newX) { x = newX; } // Cannot be const because it modifies 'x'  
};
```

Example

```
#include <iostream>
using namespace std;
class Book{
    private:
        string title;
        string author;
    public:
        Book(string t, string a){
            title = t;
            author = a; }
        string getTitle()
        {
            title = "new title";
            return title; } };

int main()
{
    Book b1("abc", "xyz");
    cout<<b1.getTitle()<<endl;
    return 0;
}
```

Example

```
#include <iostream>
using namespace std;
class Book{
    private:
        string title;
        string author;
    public:
        Book(string t, string a){
            title = t;
            author = a; }
        string getTitle() const
        {
            title = "new title"; // Compilation Error
            return title; } };

int main()
{
    const Book b1("abc", "xyz");
    cout<<b1.getTitle()<<endl;
    return 0;
}
```

Example

```
#include <iostream>
using namespace std;
class Book{
    private:
        string title;
        string author;
    public:
        Book(string t, string a){
            title = t;
            author = a; }
        string getTitle() const
        {
            return title;
        }
};

int main()
{
    const Book b1("abc", "xyz");
    cout<<b1.getTitle()<<endl;
    return 0;
}
```

Example-2

```
#include <iostream>
class Book {
public:
    const int pages; // Const data member
    int currentPage;
    // Constructor to initialize const data member
    Book(int numPages) : pages(numPages),
    currentPage(1) {}
    // Const member function: promises not to modify
    the object
    void displayPage() const {
        std::cout << "Current Page: " << currentPage
        << " of " << pages << std::endl;
        // currentPage++; // Error: cannot modify a
        member in a const function
    }
    // Non-const member function: can modify the
    object
    void nextPage() {
        currentPage++;
    }
};
```

```
int main() {
    const Book classic(500); // A const object

    classic.displayPage(); // OK: `displayPage` is a const
    function
    // classic.nextPage(); // ERROR: `classic` is const,
    cannot call a non-const function

    Book paperback(300); // A non-const object
    paperback.displayPage();
    paperback.nextPage();
    paperback.displayPage();

    return 0;
}
```

Example – Use case

Pass By Value

Secured

More space for larger variables

Pass By Reference

No overhead of copying large data

Not Secure

Key Takeaway

const on an object = *"I promise this object's data will never change."*

const on a function = *"I promise this function will never change the object it's called on."*

Static Members

Static Members

Static members are class-level members, meaning they **belong to the class** itself rather than to any specific object. There is **only one copy of a static member, shared by all objects** of that class.

- **Static data members:** A variable that is declared **static** inside a class. It must be defined (allocated memory) outside the class definition, typically in a source file. It's used for data that is common to all objects.
- **Static member functions:** A function that is declared **static**. It can be called without creating an object of the class. It can only access other static members of the class, not non-static data members, because it doesn't operate on a specific object instance.

Static Members of a Class

Concept: Members (data or functions) that belong to the class itself, rather than to any specific object of the class.

Lifetime: They exist throughout the program's execution, from the start of the program until its end.

Access: Can be accessed directly using the class name and the scope resolution operator (`ClassName::memberName`), even without creating an object of the class.

Static Data Members

Definition: A data member declared with the `static` keyword.

Characteristics:

- Only **one copy** exists for the entire class, shared by all objects of that class.
- Initialized outside the class definition, typically in the `.cpp` file.
- Can be `private`, `protected`, or `public`.

Purpose:

- To maintain a count of objects created.
- To store common data shared by all objects.

Example:

```
class Logger {  
public:  
    static int instanceCount; // Declaration inside class  
    Logger() { instanceCount++; }  
    ~Logger() { instanceCount--; }  
};  
  
int Logger::instanceCount = 0; // Definition and initialization outside
```

Static Member Functions

Definition: A member function declared with the `static` keyword.

Characteristics:

- Can be called directly using the class name (`ClassName::staticFunction()`) without an object.
- Can only access other `static` data members and `static` member functions of the same class.
- Cannot access `non-static` data members or `non-static` member functions (because they don't have a `this` pointer).

Purpose:

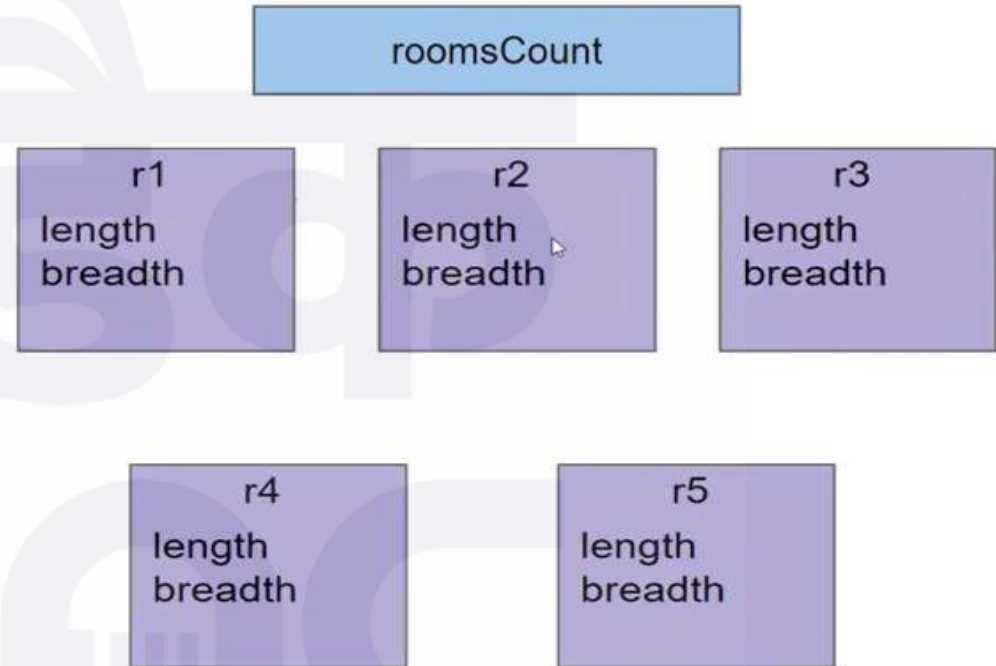
- To operate on `static` data members.
- To provide utility functions related to the class but not specific to an object.

Example:

```
class MathUtil {  
public:  
    static int add(int a, int b) { return a + b; }  
};  
// Usage: int sum = MathUtil::add(5, 3);
```

Example

```
#include <iostream>
using namespace std;
class Room{
private:
    static int roomsCount;
    int length;
    int breadth;
public:
    Room(int l, int b){
        length = l;
        breadth = b; }
    int area(){
        return length * breadth; } };
int main(){
    Room r1(10, 20);
    Room r2(15, 25);
    Room r3(30, 40);
    cout << "Area of Room: " << r1.area() << endl;
    return 0;
}
```



```
#include <iostream>
using namespace std;
class Room{
    private:
        static int roomcount;
        int length;
        int breadth;
    public:
        Room(int l, int b){
            length = l;
            breadth = b;
            roomcount++; }
        int area(){
            return length * breadth; }
        static int getRoomCount(){
            return roomcount; } };

```

```
int Room::roomcount = 0;
int main(){
    Room r1(10, 20);
    Room r2(15, 25);
    Room r3(30, 40);
    cout << "Area of Room: " << r1.area() <<
endl;
    cout << Room::getRoomCount() << " rooms
created." << endl;
    return 0;
}

```

Example-2

```
#include <iostream>

class Player {
public:
    static int playerCount; // Static data member
    declaration
    Player() {
        playerCount++; // Every time a Player is
        created, increment the count
    }
    ~Player() {
        playerCount--;
    }
    static int getPlayerCount() { // Static member
    function
        return playerCount;
    }
};
```

```
int Player::playerCount = 0; // Definition and
initialization of the static member
```

```
int main() {
    Player p1;
    Player p2;

    // Access static member via the class name
    std::cout << "Number of players: " <<
    Player::getPlayerCount() << std::endl;

    Player* p3 = new Player();

    std::cout << "Number of players: " <<
    Player::playerCount << std::endl;

    delete p3;

    std::cout << "Number of players: " <<
    Player::getPlayerCount() << std::endl;

    return 0;
}
```

Friend Functions and Classes

Friend and Classes

Concept: In C++, private and protected members of a class are typically inaccessible from outside the class.

Need for Friends: Sometimes, it's necessary for an external function or another class to access the private or protected members of a class for specific operations, while still maintaining encapsulation for general use.

Solution: Friend mechanisms provide a controlled way to grant this special access.

Friend Functions and Classes

A **friend** is a **non-member of a class** that is granted special access to its **private** and **protected** members. This is used when a function or a class needs to access the internal data of another class but should not be a member of that class.

- **Friend Function:** A function that is not a member of the class but can access its private members. It is declared with the **friend** keyword inside the class.
- **Friend Class:** A class that is declared a friend of another class. All member functions of the friend class can access the private members of the other class.

Object Communication

Standard Communication: Objects typically communicate through their public member functions (methods). This adheres to encapsulation.

When Friend Functions are Useful for Communication:

- When an operation involves two different classes and requires access to private members of both.
- When an operation is more naturally expressed as a non-member function but needs access to private data.
- For stream I/O operations (<< and >>) which are often implemented as friend functions.

Friend Functions

Definition: A non-member function that is declared as a **friend** of a class.

Access: A friend function can access the **private** and **protected** members of the class that declared it as a friend.

Declaration: Declared inside the class definition, prefixed with the **friend** keyword. It can be declared in the public, private, or protected section; its placement does not affect its access rights.

Syntax:

```
class MyClass {  
private: int privateData;  
public:  friend void myFunction(MyClass obj); // Friend function declaration  };  
void myFunction(MyClass obj) { // Can access obj.privateData }
```

Caution: Use sparingly, as they violate strict encapsulation.

Friend Classes

Definition: When one class is declared as a **friend** of another class.

Access: All member functions of the friend class can access the **private** and **protected** members of the class that declared it as a friend.

Declaration: Declared inside the class definition, prefixed with the **friend** keyword.

Syntax:

```
class ClassA {  
    private:  
        int dataA;  
    public:  
        friend class ClassB; // ClassB is a friend of ClassA    };  
class ClassB {  
    public:  
        void accessA(ClassA objA) {  
            // Can access objA.data    }    };  
};
```

Caution: Grants broad access, so use with careful design considerations.

Example

```
#include <iostream>
using namespace std;
class Count{
    private:
        int x;
    public:
        Count(): x(0)
        {
        }
        void setx(int x1);
        int getX()
        { return x; }    };
```

```
void Count::setx(int x1)
{
    x=x1;
}
int main()
{
    Count c1;
    c1.setx(10);
    cout<<"Value of x is: " << c1.getX()
    << endl;
    return 0;
}
```

Example – Contd...

```
#include <iostream>
using namespace std;
class Count{
    friend void setx(Count &c, int x1);
private:
    int x;
public:
    Count(): x(0)
    {
    }
    int getX()
    { return x; }    };
```

```
void setx(Count &c, int x1)
{
    c.x=x1;
}
int main()
{
    Count c1;
    Count c2;
    setx(c1, 10); //particular object
    setx(c2, 11);
    cout<<"Value of x for c1 is: "<< c1.getX() <<
endl;
    cout<<"Value of x for c2 is: "<< c2.getX() <<
endl;
    return 0; }
```

```
#include <iostream>

class MyClass {
private:
    int private_data;
public:
    MyClass() : private_data(10) {}
    // Declaration of a friend function
    friend void friendFunction(MyClass &obj);
    // Declaration of a friend class
    friend class FriendClass;
};

// Friend function definition
void friendFunction(MyClass &obj) {
    std::cout << "Friend function can access private
data: " << obj.private_data << std::endl;
}
```

```
// Friend class definition
class FriendClass {
public:
    void accessPrivateData(MyClass &obj) {
        std::cout << "Friend class can access private
data: " << obj.private_data << std::endl;
    }
};

int main() {
    MyClass obj;
    friendFunction(obj);
    FriendClass friendObj;
    friendObj.accessPrivateData(obj);
    return 0;
}
```



This Pointer

This Pointer

Definition: A **special, implicit pointer** available inside non-static member functions of a class.

Value: Points to the current object on which the member function is being called.

Purpose:

- To refer to the current object's members when there's ambiguity (e.g., parameter name same as data member).
- To return a reference or pointer to the current object (e.g., in operator overloading for cascading).
- To pass the current object to another function.

Example:

```
class Point {  
    int x, y;  
public:  
    Point(int x, int y) : x(x), y(y) {} // 'this->x' is implicit  
    Point& setX(int val) {  
        this->x = val; // Explicitly use this->  
        return *this; // Return reference to current object  
    }  
};
```

Example

```
#include <iostream>
using namespace std;
class Message{
    private:
        string from;
        string to;
        string body;
    public:
        void setFrom(string from){
            from = from;
        };
};
int main(){
    Message m;
    m.setFrom("Alice");
    return 0; }
```

```
#include <iostream>
using namespace std;
class Message{
    private:
        string from;
        string to;
        string body;
    public:
        void setFrom(string from){
            this->from = from; }
        string getFrom(){
            return from; } };
int main(){
    Message m;
    m.setFrom("Alice");
    cout << m.getFrom() << endl;
    return 0; }
```



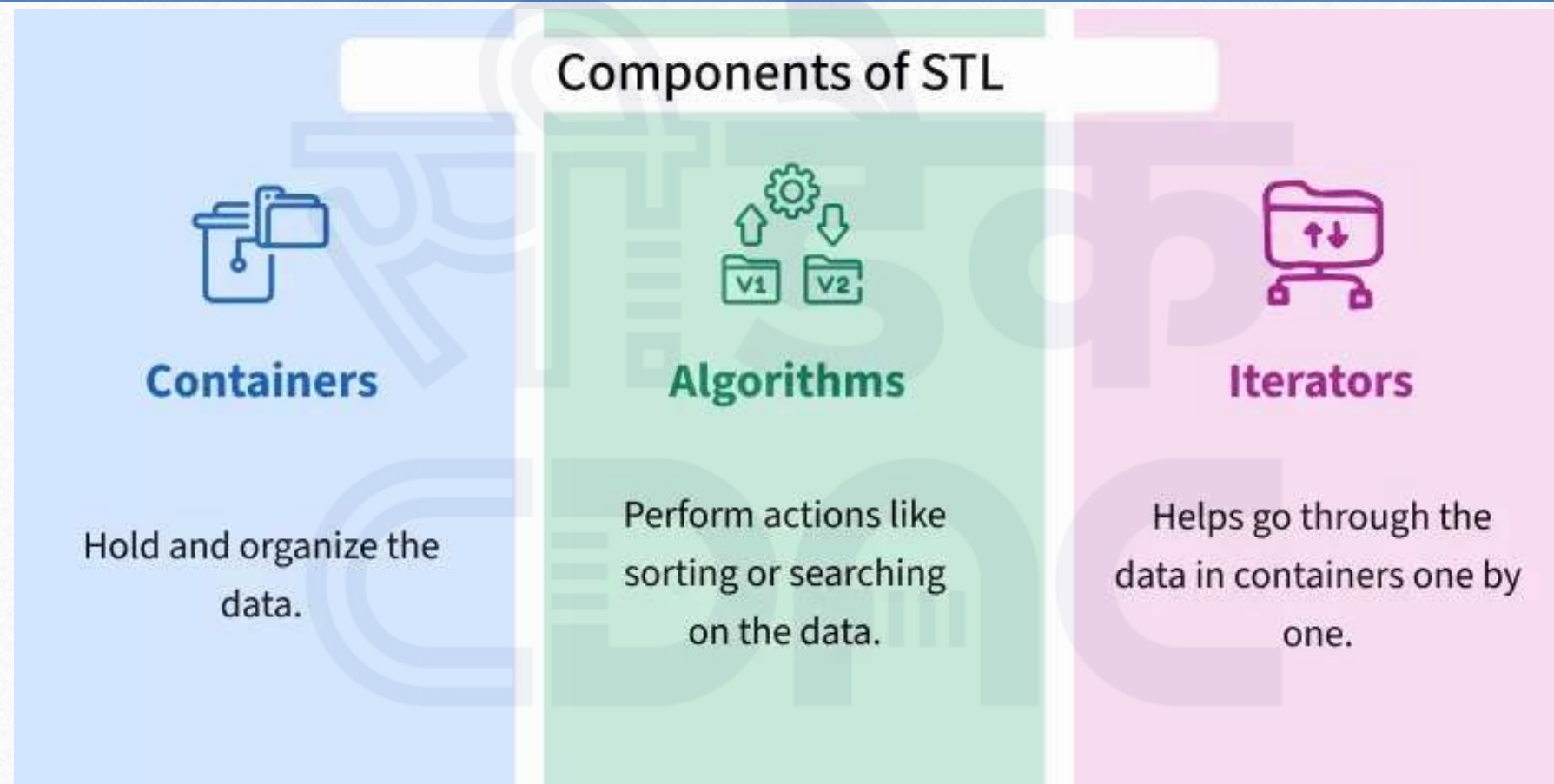
STL

Containers and Iterators

Standard Template Library (STL)

- STL (Standard Template Library) is a **collection of pre-built classes and functions** in C++ used for efficient data handling and programming. It provides ready-to-use data structures and algorithms that simplify development.
- Includes containers like vectors, stacks, queues, and maps.
- Saves time and improves performance using reusable components.

Components of STL



Containers

Containers are the data structures used to store objects and data according to the requirement. Each container is implemented as a template class that also contains the methods to perform basic operations on it. Every STL container is defined inside its own header file.

Containers can be further classified into 4 types:

Sequence Containers : Vector, Deque, List, Forward List, Array

Container Adaptors : Stack, Queue, Priority Queue

Associative Containers : Set, Multiset, Map, Multimap

Unordered Associative Containers : Unordered Set, Unordered Multiset, Unordered Map, Unordered Multimap

Algorithms

STL algorithms offer a wide range of functions to perform common operations on data (mainly containers).

These functions implement the most efficient version of the algorithm for tasks such as sorting, searching, modifying and manipulating data in containers, etc.

Most STL algorithms are defined in `<algorithm>` and `<numeric>`, while some specialized algorithms and utilities are available in other headers like `<memory>`, `<functional>`, and `<iterator>`.

Iterators

Iterators provide a common way to access and traverse elements stored in STL containers.

Act as pointer-like objects for accessing container elements.

Connect STL containers with generic algorithms.

Support operations such as traversing, accessing, and modifying elements.

Different containers provide different types of iterators based on their capabilities.

Vectors

- A vector in C++ is a dynamic sequence container provided by the Standard Template Library (STL). It stores elements in contiguous memory and automatically manages its storage as elements are added or removed.
- Vectors can dynamically grow or shrink during program execution.
- They provide constant-time random access and support efficient insertion and deletion at the end.

Declaration and Initialization

- The vector class template is defined in the `<vector>` header file.

`vector<T> v;`

- T is the data type of the elements.
- v is the name of the vector.

Example 1 – Print Elements in Vector using **Range-based For Loop**

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = {2, 4, 6, 8, 10};
    v.push_back(12);
    v.push_back(14);
    v.pop_back();
    for (int x: v) {
        cout << x << " ";
    }
}
```

Example 2- Methods to Access **Vector** Elements

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v = {2, 4, 6, 8, 10};
    v.push_back(12);
    v.push_back(14);
    v.pop_back();

    vector<int>::iterator itr;
```

```
    cout<<"Using iterator: "<<endl;
    for ( itr = v.begin(); itr != v.end(); itr++)
    {
        cout << *itr << " ";
    }
    cout<<endl;
```

```
    cout<<"Using range-based for loop: " <<
endl;
    for (int x: v) {
        cout << x << " ";
    }
}
```

Example 3- Methods to Access **List** Elements

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> v = {2, 4, 6, 8, 10};
    v.push_back(12);
    v.push_back(14);
    v.pop_back();

    list<int>::iterator itr;
```

```
    cout<<"Using iterator: "<<endl;
    for ( itr = v.begin(); itr != v.end(); itr++)
    {
        cout << *itr << " ";
    }
    cout<<endl;
```

```
    cout<<"Using range-based for loop: " <<
endl;
    for (int x: v) {
        cout << x << " ";
    }
}
```

Example 4- Methods to Access **Forward_List** Elements

```
#include <iostream>
#include <forward_list>
using namespace std;
int main() {
    forward_list<int> v = {2, 4, 6, 8, 10};
    v.push_front(12);
    v.push_front(14);
    v.pop_front();

    forward_list<int>::iterator itr;

    cout<<"Using iterator: "<<endl;
    for ( itr = v.begin(); itr != v.end(); itr++)
    {
        cout << *itr << " ";
    }
    cout<<endl;

    cout<<"Using range-based for loop: " <<
    endl;
    for (int x: v) {
        cout << x << " ";
    }
}
```

Example 5- Methods to Access **Set** Elements

```
#include <iostream>
#include <set>
using namespace std;
int main() {
    set<int> v = {2, 4, 6, 8, 10,10};
    v.insert(12);
    v.insert(14);
    v.erase(6);

    set<int>::iterator itr;
```

```
    cout<<"Using iterator: "<<endl;
    for ( itr = v.begin(); itr != v.end(); itr++)
    {
        cout << *itr << " ";
    }
    cout<<endl;
```

```
    cout<<"Using range-based for loop: " <<
endl;
    for (int x: v) {
        cout << x << " ";
    }
}
```

Example 5- Methods to Access **Map** Elements

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, string> m;
    m.insert(pair<int, string>(1, "Alice"));
    m.insert(pair<int, string>(2, "Bob"));
    m.insert(pair<int, string>(3, "Charlie"));

    map<int, string>::iterator itr;
    for (itr = m.begin(); itr != m.end(); itr++) {
        cout << itr->first << " ==> " << itr->
second << endl; }

    map<int, string>::iterator itr1;
    itr1 = m.find(2);
    if (itr1 != m.end()) {
        cout << "Found: " << itr1-> first
<< " ==> " << itr1-> second << endl;
    } else {
        cout << "Not found" << endl;
    }
}
```

Task

- Create grocery list using vector

Templates and Generics

Templates

Templates are a powerful feature in C++ that allow you to write generic code that works with any data type. This is known as **generic programming**. Instead of writing separate code for `int`, `double`, and `string`, you can write a single template that works with all of them.

Function and Class Templates

A **function template** is a blueprint for a function. The compiler generates an actual function from the template for each data type it is called with.

Similarly, a **class template** is a blueprint for a class, allowing you to define a class that operates on generic data types.

Example

```
#include <iostream>
// This is a class template for a simple 'Pair'
template <typename T1, typename T2>
class Pair {
public:
    T1 first;
    T2 second;
    Pair(T1 f, T2 s) : first(f), second(s) {}
    void print_pair() {
        std::cout << "First: " << first << ",
Second: " << second << std::endl;
    }
};
```

```
int main() {
    // Instantiates a Pair class for an int and a
    double
    Pair<int, double> p1(10, 20.5);
    p1.print_pair(); // Output: First: 10, Second:
20.5

    // Instantiates a separate Pair class for two
    strings
    Pair<std::string, std::string> p2("Hello",
"World");
    p2.print_pair(); // Output: First: Hello,
Second: World

    return 0;
}
```

Template Specialization

Sometimes, a general template may not be the most efficient or correct solution for a specific data type. **Template specialization** allows you to provide a custom implementation for a particular type.

- **Full Specialization:** Provides a unique, non-templated implementation for a specific type.
- **Partial Specialization:** Provides a unique implementation for a subset of template arguments.

```

#include <iostream>
// General function template
template <typename T>
void print_type(T val) {
    std::cout << "General template for a
simple value." << std::endl; }
// Full specialization for a pointer type
template <>
void print_type<int*>(int* val) {
    std::cout << "Specialized for int*,
value: " << *val << std::endl; }
// Class template with partial
specialization
template <typename T1, typename T2>
class Pair {
public:
    Pair(T1 v1, T2 v2) {
        std::cout << "General Pair
template." << std::endl; }
};

```

```

// Partial specialization for when both
types are the same
template <typename T>
class Pair<T, T> {
public:
    Pair(T v1, T v2) {
        std::cout << "Partially specialized
Pair template for same types." << std::endl;
    }
};
int main() {
    int x = 10;
    print_type(x);
    print_type(&x); // Calls the full
specialization
    Pair<int, double> p1(1, 2.5); // Calls
general Pair template
    Pair<int, int> p2(1, 2); // Calls
partially specialized Pair template
    return 0; }

```

Variadic Templates

Variadic templates are templates that can take a variable number of arguments.

This is incredibly useful for writing functions that can accept any number of inputs, like a print function or a function to sum a list of numbers.

The arguments are bundled into a **parameter pack**. You use a recursive approach to process each element of the pack.

Example - Variable Parameter List (Sum)

```
#include<iostream>
#include<cstdarg>
using namespace std;
int sum(int count, ...) {
    va_list args;
    va_start(args, count);
    int sum = 0;
    for (int i = 0; i < count; ++i) {
        sum += va_arg(args, int);
    }
    va_end(args);
    return sum;
}

int main() {
    int result = sum(3, 1, 2, 3);
    cout << "The sum is: " <<
    result << endl;
    return 0;
}
```

Variadic Templates

```
#include <iostream>

// Base case to end the recursion
void print() {
    std::cout << std::endl; }

// Variadic function template
template <typename T, typename... Args>
void print(T first_arg, Args... remaining_args) {
    std::cout << first_arg << " ";
    print(remaining_args...); // Recursively call print
                              // with the rest of the pack
}
```

```
int main() {
    print(1);
    print(1, 2, 3);
    print("hello", 10.5, 'a');
    return 0;
}
```

Example

```
#include <iostream>
// Base Case
// Handles the last argument and its return type
template<typename T>
T sum_recursive(T arg) {
    return arg; }
// Recursive Variadic Case
// Deduces the return type by adding the current argument to the result of the recursive call
template<typename T, typename... Args>
auto sum_recursive(T arg, Args... args) {
    return arg + sum_recursive(args...); }
int main() {
    auto result = sum_recursive(1, 2.5, 3.14f);
    std::cout << "Result: " << result << std::endl; // The return type is double
    return 0; }
```